add, coupled with the typically bespoke nature of individual Razor Pages, means that I normally find they aren't worth using. Your mileage may vary, of course, but don't jump to them as a first option.

That brings us to the end of this detailed look at each of the filters in the MVC pipeline. Looking back and comparing listings 22.1 and 22.2, you can see filters allowed us to refactor the controllers and make the intent of each action method much clearer. Writing your code in this way makes it easier to reason about, as each filter and action has a single responsibility.

In the next section we'll take a slight detour into exactly what happens when you short-circuit a filter. I've described *how* to do this, by setting the `context.Result` property on a filter, but I haven't described exactly what happens. For example, what if there are multiple filters in the stage when it's short-circuited? Do those still run?

22.2 Understanding pipeline short-circuiting

In this short section you'll learn about the details of filter-pipeline short-circuiting. You'll see what happens to the other filters in a stage when the pipeline is short-circuited and how to short-circuit each type of filter.

A brief warning: the topic of filter short-circuiting can be a little confusing. Unlike middleware short-circuiting, which is cut-and-dried, the filter pipeline is a bit more nuanced. Luckily, you won't often need to dig into it, but when you do, you'll be glad for the detail.

You short-circuit the authorization, resource, action, page, and result filters by setting `context.Result` to `IActionResult`. Setting an action result in this way causes some or all of the remaining pipeline to be bypassed. But the filter pipeline isn't entirely linear, as you saw in chapter